

# Sudoku genérico como CSP

## Introdução

O Sudoku clássico é um exemplo canónico de **problema de satisfação de restrições (CSP)**: uma grelha  $n^2 \times n^2$  onde cada linha, cada coluna e cada bloco  $n \times n$  tem de conter os valores de 1 a  $n^2$  sem repetições. A regra é sempre a mesma ("todos diferentes"); o que muda de grupo para grupo é apenas **que células** o compõem. Neste trabalho modelamos o problema com uma abstração única, a classe `box`, que representa um conjunto de células, algumas possivelmente fixas a um valor. Linhas, colunas e blocos são casos particulares ( `path` e `cube` ), e as pistas aleatórias são, outra vez, uma `box`. O modelo recebe um número arbitrário de grupos e trata-os todos da mesma forma, sem saber de onde vieram.

## Escolha do solver

Começamos por resolver o problema com o **Z3**, através do pySMT, porque foi o solver que aprendemos nas aulas e aquele com que nos sentíamos mais à vontade. Mais tarde aprendemos a usar o **CP-SAT** do OR-Tools, e adaptamos o código que já tínhamos para o Z3. Como a classe `box` e os grupos não dependem do solver (só o modelo fala com a biblioteca), a adaptação limitou-se à classe do modelo, e pudemos comparar as duas abordagens.

## Imports

```
In [ ]: import marimo as mo
        from pysmt.shortcuts import Symbol, Solver, And, LE, GE, Int, Equals,
        AllDifferent
        from pysmt.typing import INT
        import random
        from ortools.sat.python import cp_model
        import time
```

## Classes

### Classe box (R1)

A classe `box` é uma classe genérica que representa um conjunto de células de uma grelha  $n^2 \times n^2$ . Esta classe possui os métodos:

- `add(self, i, j, val=None)`, que acrescenta um certo valor de 1 a  $n^2$  ou None à grelha, dentro das coordenadas (i,j), com i,j pertencentes a  $[0, n^2-1]$ . Caso as coordenadas ou o valor sejam inválidos, este é rejeitado.
- `to_matrix(self)`, que converte o dicionário `cells` numa matriz  $n^2 * n^2$ . Caso o valor com chave (i,j) não existe em `cells`, é colocado um 0 no lugar.

```
In [ ]: # R1
class box:

    def __init__(self, n, cells=None):
        self.n = n
        self.size = n * n
        self.cells = {}
        for (i, j), val in (cells or {}).items():
            self.add(i, j, val)

    def add(self, i, j, val=None):
        if not (0 <= i < self.size and 0 <= j < self.size):
            raise ValueError(f"Célula ({i}, {j}) fora da grelha {self.size}x{self.size}")
        if val is not None and not (1 <= val <= self.size):
            raise ValueError(f"Valor {val} fora de [1, {self.size}]")
        self.cells[(i, j)] = val

    def to_matrix(self):
        m = [[0] * self.size for _ in range(self.size)]
        for (i, j), val in self.cells.items():
            if val is not None:
                m[i][j] = val
            else:
                m[i][j] = 0
        return m
```

## Classe cube (R2)

Um grupo que representa o bloco  $i \times j$  cujo canto superior esquerdo é a célula ( $i \times n, j \times n$ ), parametrizado pelos índices de bloco (i,j) com  $0 \leq i, j < n$ .

```
In [ ]: # R2
class cube(box):

    def __init__(self, n, bi, bj):
        if not (0 <= bi < n and 0 <= bj < n):
            raise ValueError(f"Índices de cube fora de [0, {n-1}]")
        super().__init__(n)
        for di in range(n):
            for dj in range(n):
                self.add(bi * n + di, bj * n + dj)
```

## Classe path (R3)

Um grupo que representa o troço reto (horizontal ou vertical) de células entre duas coordenadas inicio e fim, inclusive. Não representa necessariamente uma linha ou coluna, podendo apenas representar um segmento mais pequeno.

```
In [ ]: # R3
class path(box):

    def __init__(self, n, inicio, fim):
        super().__init__(n)
        (i0, j0), (i1, j1) = inicio, fim
        if i0 != i1 and j0 != j1:
            raise ValueError("inicio e fim têm de estar na mesma linha ou
coluna")
        di = (i1 > i0) - (i1 < i0)
        dj = (j1 > j0) - (j1 < j0)
        i, j = i0, j0
        self.add(i, j)
        while (i, j) != (i1, j1):
            i, j = i + di, j + dj
            self.add(i, j)
```

## Classe model (R5)

Uma classe que aceita dois solvers, z3 e CP-SAT (CP-SAT por defeito).

Esta classe possui os métodos:

- `add_groups(self, boxes)`, que recebe um número arbitrário de grupos (box, cube, path, ou pistas aleatórias) e, para cada um, impões que as suas células sejam todas diferentes e fixa as que tiverem valor atribuído, neste problema apenas atribuiremos valores às pistas aleatórias.
- `solve(self)`, que devolve um sudoku resolvido caso haja solução, ou None caso contrário.
- `imprime_box_aleatorio(self, N)`, que imprime a pista aleatória, o ponto de partida para a resolução do sudoku.

```
In [ ]: # R5
class model:
    def __init__(self, n, solver=""):
        self.size = n * n
        self.box_inicial = None
        self.solver_name = solver

        if self.solver_name == "z3":
            self.solver = Solver(name="z3")
            self.x = {(i, j): Symbol(f"x_{i}_{j}", INT) for i in
range(self.size) for j in range(self.size)}
            for s in self.x.values():
                self.solver.add_assertion(And(GE(s, Int(1)), LE(s,
Int(self.size))))
        else:
```

```

self.model = cp_model.CpModel()
self.solver = cp_model.CpSolver()
self.x = {
    (i, j): self.model.NewIntVar(1, self.size, f"x_{i}_{j}")
    for i in range(self.size)
    for j in range(self.size)
}

def add_groups(self, boxes):
    for b in boxes:
        if self.solver_name == "z3":
            if isinstance(b, (path, cube)):
                self.solver.add_assertion(AllDifferent([self.x[c] for c
in b.cells]))
            else:
                self.box_inicial = b
                for c, val in b.cells.items():
                    if val is not None:
                        self.solver.add_assertion(Equals(self.x[c],
Int(val)))
        else:
            if isinstance(b, (path, cube)):
                self.model.AddAllDifferent([self.x[c] for c in
b.cells])
            else:
                self.box_inicial = b
                for c, val in b.cells.items():
                    if val is not None:
                        self.model.Add(self.x[c] == val)

def solve(self):
    if self.solver_name == "z3":
        if not self.solver.solve():
            return None
        return [[self.solver.get_value(self.x[(i, j)]).constant_value()
for j in range(self.size)] for i in range(self.size)]
    else:
        status = self.solver.Solve(self.model)
        if status in (cp_model.OPTIMAL, cp_model.FEASIBLE):
            return [[self.solver.Value(self.x[(i,j)]) for j in
range(self.size)] for i in range(self.size)]
        else:
            return None

def imprime_box_aleatorio(self, N):
    print("Pista aleatória")
    print_matriz(N, self.box_inicial.to_matrix())
    print()

```

## Funções

### Geração Aleatória de pistas (R4)

## def box\_aleatorio(n, k=None)

Uma função que devolve um grupo box, com k células preenchidas ou n, caso k não seja fornecido. Devolve um box completamente aleatório, podendo este ser inválido para a resolução do sudoku.

```
In [ ]: # R4
def box_aleatorio(n, k=None):
    b = box(n)
    rep = k if k is not None else n
    todas_coord = [(i,j) for i in range(b.size) for j in range(b.size)]
    for i, j in random.sample(todas_coord, rep):
        b.add(i, j, random.randint(1,b.size))
    return b
```

## Resolução (R6)

### def criar\_sudoku(n)

Uma função que retorna um grupo de  $n^2$  linhas,  $n^2$  colunas,  $n^2$  de cubes e uma pista aleatória.

```
In [ ]: # R6
def criar_sudoku(n):
    s = n * n
    linhas = [path(n, (i, 0), (i, s - 1)) for i in range(s)]
    colunas = [path(n, (0, j), (s - 1, j)) for j in range(s)]
    blocos = [cube(n, i, j) for i in range(n) for j in range(n)]
    return linhas + colunas + blocos + [box_aleatorio(n)]
```

## Verificação

### def verificar\_sudoku(n, sudoku, box)

Função que verifica se todos os elementos das linhas, colunas e cubes são diferentes e que os elementos da pista aleatória inicial estão presentes na matriz, com os valores e coordenadas iguais.

```
In [ ]: def verificar_sudoku(n, sudoku, box):
    ok = True
    size = n*n
    # verificar linhas e colunas
    for c in range(size):
        linhas = []
        colunas = []
        for d in range(size):
            # linhas
            if (co := sudoku[c][d]) in linhas:
                print(f"Valor {co} repetido na linha {c}")
                ok = False
```

```

        linhas.append(co)

    # colunas
    if (li := sudoku[d][c]) in colunas:
        print(f"Valor {li} repetido na coluna {c}")
        ok = False
    colunas.append(li)

# cubes
for c in range(n):
    for d in range(n):
        cube = []
        for e in range(n):
            for f in range(n):
                if (val := sudoku[e + c*n][f + d*n]) in cube:
                    print(f"Valor {val} repetido no cube {c*n + d}")
                    ok = False
                cube.append(val)

# verificar valores do box inicial
for (i,j), val in box.cells.items():
    if val != (atual := sudoku[i][j]):
        print(f"Valor inicial {val} nas coordenadas ({i},{j}) alterado para {atual}")
        ok = False

# verificar se os valores do sudoku são válidos
for c in range(size):
    for d in range(size):
        if not 1 <= (val := sudoku[c][d]) <= size:
            print(f"Valor {val} inválido em ({c},{d}")
            ok = False

return ok

```

**def print\_matriz(n, m)**

Imprime o sudoku  $n^2 * n^2$

```

In [ ]: def print_matriz(n, m):
        w = len(str(n*n))
        for c in range(n*n):
            for d in range(n):
                print(' ', end='')
                for e in range(n):
                    print(f"{m[c][d * n + e]:{w}}", end=' ')
                if (d + 1 != n):
                    print('|', end='')
            if (c + 1) % n == 0 and c + 1 < n * n:
                print('\n' + '_' * ((n+1)*n + (n-1) + (n*n*w)))
            else:
                print()

```

## Uso de LLM

Neste trabalho, usamos o Claude e o ChatGPT como apoio neste trabalho. Em concreto:

- Explicação de classes e herança em Python, de \*args, de listas por compreensão e do pySMT.
- Discussão de solvers (CP-SAT, Z3 e SCIP) e escolha do Z3 com o pySMT, por ser o que a UC ensina e do CP-SAT por ser o que a UC recomenda.
- Código: o LLM propôs a estrutura inicial das classes box, cube, path e model, e de funções auxiliares. O código foi todo revisado, adaptado para o trabalho e aprovado pelo grupo. A função print\_matriz foi feita inteiramente pelo grupo. As restantes funções foram adaptadas ao problema.
- Ajuda a perceber erros (por exemplo, o \* ao passar a lista de grupos, e a chamada do método na classe em vez do objeto).
- O Chat-GPT apenas foi usado para explicar as funções do CP-SAT.
- Escrita deste relatório.

[link para o chat do claude](#)

[link para o chat do Chat-GPT](#)

## Testes

### CP-SAT (n=2)

```
In [ ]: def _():
    inicio = time.perf_counter()
    N = 2
    m = model(N)
    m.add_groups(criar_sudoku(N))
    m.imprime_box_aleatorio(N)
    sudoku = m.solve()
    if sudoku is not None:
        print("Solução")
        print_matriz(N, sudoku)
        if verificar_sudoku(N, sudoku, m.box_inicial):
            print("Sudoku sem valores repetidos")
            fim = time.perf_counter()
            print(f"Tempo de execução: {fim - inicio:.4f}s")
    else:
        print("Sem solução")
    return
_()
```

Pista aleatória

|   |   |  |   |   |
|---|---|--|---|---|
| 0 | 0 |  | 0 | 0 |
| 0 | 3 |  | 0 | 0 |

---

|   |   |  |   |   |
|---|---|--|---|---|
| 0 | 0 |  | 0 | 0 |
| 0 | 0 |  | 0 | 4 |

Solução

|   |   |  |   |   |
|---|---|--|---|---|
| 4 | 2 |  | 1 | 3 |
| 1 | 3 |  | 4 | 2 |

---

|   |   |  |   |   |
|---|---|--|---|---|
| 3 | 4 |  | 2 | 1 |
| 2 | 1 |  | 3 | 4 |

Sudoku sem valores repetidos

Tempo de execução: 0.0097s

## CP-SAT (n=3)

```
In [ ]: box_alea = box_aleatorio(3)
def _():
    inicio = time.perf_counter()
    N = 3
    m = model(N)
    m.add_groups(criar_sudoku(N)[: -1] + [box_alea])
    m.imprime_box_aleatorio(N)
    sudoku = m.solve()
    if sudoku is not None:
        print("Solução")
        print_matriz(N, sudoku)
        if verificar_sudoku(N, sudoku, m.box_inicial):
            print("Sudoku sem valores repetidos")
            fim = time.perf_counter()
            print(f"Tempo de execução: {fim - inicio:.4f}s")
    else:
        print("Sem solução")
    return
_()
```

Pista aleatória

|   |   |   |  |   |   |   |  |   |   |   |
|---|---|---|--|---|---|---|--|---|---|---|
| 0 | 0 | 0 |  | 0 | 0 | 0 |  | 0 | 0 | 0 |
| 0 | 0 | 0 |  | 0 | 0 | 0 |  | 0 | 0 | 0 |
| 0 | 0 | 0 |  | 0 | 0 | 0 |  | 0 | 0 | 0 |

|   |   |   |  |   |   |   |  |   |   |   |
|---|---|---|--|---|---|---|--|---|---|---|
| 0 | 0 | 0 |  | 0 | 0 | 0 |  | 0 | 0 | 0 |
| 0 | 0 | 0 |  | 0 | 0 | 0 |  | 0 | 0 | 4 |
| 0 | 0 | 0 |  | 0 | 0 | 0 |  | 0 | 0 | 0 |

|   |   |   |  |   |   |   |  |   |   |   |
|---|---|---|--|---|---|---|--|---|---|---|
| 0 | 0 | 0 |  | 0 | 0 | 0 |  | 0 | 0 | 0 |
| 0 | 0 | 0 |  | 0 | 0 | 5 |  | 0 | 0 | 0 |
| 0 | 0 | 0 |  | 0 | 4 | 0 |  | 0 | 0 | 0 |

Solução

|   |   |   |  |   |   |   |  |   |   |   |
|---|---|---|--|---|---|---|--|---|---|---|
| 1 | 3 | 2 |  | 8 | 5 | 7 |  | 4 | 6 | 9 |
| 5 | 7 | 8 |  | 4 | 6 | 9 |  | 1 | 2 | 3 |
| 9 | 4 | 6 |  | 1 | 2 | 3 |  | 7 | 8 | 5 |

|   |   |   |  |   |   |   |  |   |   |   |
|---|---|---|--|---|---|---|--|---|---|---|
| 4 | 5 | 7 |  | 3 | 9 | 8 |  | 2 | 1 | 6 |
| 8 | 1 | 3 |  | 6 | 7 | 2 |  | 5 | 9 | 4 |
| 2 | 6 | 9 |  | 5 | 1 | 4 |  | 8 | 3 | 7 |

|   |   |   |  |   |   |   |  |   |   |   |
|---|---|---|--|---|---|---|--|---|---|---|
| 3 | 2 | 4 |  | 7 | 8 | 6 |  | 9 | 5 | 1 |
| 7 | 9 | 1 |  | 2 | 3 | 5 |  | 6 | 4 | 8 |
| 6 | 8 | 5 |  | 9 | 4 | 1 |  | 3 | 7 | 2 |

Sudoku sem valores repetidos

Tempo de execução: 0.0895s

## Z3 (n=3)

```
In [ ]: def _():
    inicio = time.perf_counter()
    N = 3
    m = model(N, "z3")
    m.add_groups(criar_sudoku(N)[: -1] + [box_alea])
    m.imprime_box_aleatorio(N)
    sudoku = m.solve()
    if sudoku is not None:
        print("Solução")
        print_matriz(N, sudoku)
        if verificar_sudoku(N, sudoku, m.box_inicial):
            print("Sudoku sem valores repetidos")
            fim = time.perf_counter()
            print(f"Tempo de execução: {fim - inicio:.4f}s")
    else:
        print("Sem solução")
    return
_()
```

Pista aleatória

|   |   |   |  |   |   |   |  |   |   |   |
|---|---|---|--|---|---|---|--|---|---|---|
| 0 | 0 | 0 |  | 0 | 0 | 0 |  | 0 | 0 | 0 |
| 0 | 0 | 0 |  | 0 | 0 | 0 |  | 0 | 0 | 0 |
| 0 | 0 | 0 |  | 0 | 0 | 0 |  | 0 | 0 | 0 |

|   |   |   |  |   |   |   |  |   |   |   |
|---|---|---|--|---|---|---|--|---|---|---|
| 0 | 0 | 0 |  | 0 | 0 | 0 |  | 0 | 0 | 0 |
| 0 | 0 | 0 |  | 0 | 0 | 0 |  | 0 | 0 | 4 |
| 0 | 0 | 0 |  | 0 | 0 | 0 |  | 0 | 0 | 0 |

|   |   |   |  |   |   |   |  |   |   |   |
|---|---|---|--|---|---|---|--|---|---|---|
| 0 | 0 | 0 |  | 0 | 0 | 0 |  | 0 | 0 | 0 |
| 0 | 0 | 0 |  | 0 | 0 | 5 |  | 0 | 0 | 0 |
| 0 | 0 | 0 |  | 0 | 4 | 0 |  | 0 | 0 | 0 |

Solução

|   |   |   |  |   |   |   |  |   |   |   |
|---|---|---|--|---|---|---|--|---|---|---|
| 7 | 5 | 3 |  | 4 | 6 | 2 |  | 9 | 8 | 1 |
| 6 | 9 | 4 |  | 7 | 8 | 1 |  | 2 | 5 | 3 |
| 2 | 8 | 1 |  | 3 | 5 | 9 |  | 6 | 4 | 7 |

|   |   |   |  |   |   |   |  |   |   |   |
|---|---|---|--|---|---|---|--|---|---|---|
| 3 | 4 | 6 |  | 8 | 2 | 7 |  | 1 | 9 | 5 |
| 5 | 1 | 8 |  | 6 | 9 | 3 |  | 7 | 2 | 4 |
| 9 | 7 | 2 |  | 5 | 1 | 4 |  | 3 | 6 | 8 |

|   |   |   |  |   |   |   |  |   |   |   |
|---|---|---|--|---|---|---|--|---|---|---|
| 4 | 6 | 7 |  | 2 | 3 | 8 |  | 5 | 1 | 9 |
| 8 | 2 | 9 |  | 1 | 7 | 5 |  | 4 | 3 | 6 |
| 1 | 3 | 5 |  | 9 | 4 | 6 |  | 8 | 7 | 2 |

Sudoku sem valores repetidos

Tempo de execução: 25.1458s

## CP-SAT (n=7)

```
In [ ]: def _():
    inicio = time.perf_counter()
    N = 7
    m = model(N)
    m.add_groups(criar_sudoku(N))
    m.imprime_box_aleatorio(N)
    sudoku = m.solve()
    if sudoku is not None:
        print("Solução")
        print_matriz(N, sudoku)
        if verificar_sudoku(N, sudoku, m.box_inicial):
            print("Sudoku sem valores repetidos")
            fim = time.perf_counter()
            print(f"Tempo de execução: {fim - inicio:.4f}s")
    else:
        print("Sem solução")
    return
_()
```

[illegible][illegible]

---

[illegible]





19 3 38 6 39 14 4 | 48 16 30 7 33 29 5 | 1 46 41 42 24 35 31 | 25 21 2  
3 34 49 26 44 | 11 40 43 20 28 2 17 | 10 47 13 36 37 22 27 | 8 18 45 12 15  
9 32  
16 10 49 9 28 22 44 | 40 38 6 2 45 36 31 | 15 30 5 8 33 11 29 | 18 13 2  
7 39 43 3 17 | 26 32 47 19 42 25 12 | 41 7 20 46 35 24 34 | 23 21 48 4 1  
14 37  
34 20 1 17 13 15 18 | 49 41 43 22 32 27 11 | 6 14 7 3 40 37 25 | 4 12 4  
5 35 16 38 36 | 31 39 21 29 23 33 46 | 48 5 8 26 9 30 2 | 44 28 42 47 10  
19 24  
8 2 45 48 32 40 21 | 14 26 1 44 17 46 47 | 20 34 10 4 13 12 36 | 37 5 4  
2 41 6 19 24 | 16 35 15 30 38 9 22 | 28 43 23 33 25 31 18 | 3 49 27 11 7  
29 39  
29 37 5 31 23 35 47 | 9 18 25 24 42 34 19 | 21 28 39 2 38 17 44 | 40 10 1  
1 8 32 46 33 | 45 41 7 6 48 27 13 | 15 14 49 12 1 3 4 | 43 30 16 36 22  
20 26  
33 27 12 30 7 41 46 | 28 23 8 35 10 21 15 | 48 26 18 43 49 16 32 | 22 47  
1 29 14 9 20 | 44 4 24 34 37 3 36 | 19 17 6 39 42 11 45 | 40 2 38 13 31  
5 25  
42 43 36 24 26 11 25 | 13 12 3 4 37 20 39 | 23 9 22 47 19 27 45 | 7 28  
2 30 48 15 31 | 1 49 8 10 14 5 18 | 29 38 44 32 40 16 21 | 46 35 6 33 41  
34 17

---

---

46 15 17 2 16 39 12 | 21 13 41 29 38 4 49 | 27 25 9 44 32 20 48 | 14 33 2  
2 3 31 30 18 | 8 5 28 42 47 45 10 | 7 6 34 37 19 40 35 | 24 23 43 1 26  
11 36  
23 31 33 43 44 1 36 | 45 46 48 28 11 37 9 | 14 21 38 17 34 2 8 | 6 16 1  
5 20 10 13 25 | 40 22 32 26 19 35 24 | 12 41 30 5 3 49 42 | 18 7 4 39 29  
27 47  
28 13 37 7 27 38 30 | 26 3 19 39 47 32 25 | 33 1 40 49 42 31 10 | 34 41 1  
2 5 29 43 8 | 9 21 23 48 46 36 44 | 24 2 22 4 18 45 11 | 17 16 15 6 14  
35 20  
45 48 18 47 34 4 40 | 6 31 20 43 24 14 35 | 26 7 28 11 5 30 19 | 38 46 4  
4 37 36 2 32 | 33 1 3 15 49 41 27 | 23 39 17 16 29 9 13 | 12 42 8 25 21  
10 22  
14 24 26 19 9 32 6 | 5 33 15 42 8 10 17 | 36 39 47 35 23 22 18 | 21 27  
4 49 45 40 28 | 2 29 16 7 34 12 11 | 20 46 1 25 48 44 43 | 30 31 41 37 13  
38 3  
49 21 41 35 5 20 3 | 7 1 27 36 22 44 18 | 46 12 37 6 15 29 16 | 39 11 1  
7 24 47 23 42 | 13 43 30 38 31 4 25 | 32 10 26 8 28 33 14 | 19 48 40 2 9  
45 34  
22 11 8 25 29 42 10 | 12 34 40 30 2 16 23 | 41 4 45 13 3 43 24 | 48 26  
9 19 7 35 1 | 37 17 18 14 6 20 39 | 31 36 27 21 47 15 38 | 5 44 28 32 49  
33 46

---

---

27 26 31 45 33 21 41 | 34 40 9 37 14 39 13 | 17 24 19 30 7 46 6 | 28 2  
8 12 25 5 48 | 3 42 36 49 16 15 47 | 44 35 43 11 20 18 23 | 10 1 32 29 38  
22 4  
30 4 40 12 15 5 11 | 42 17 31 32 49 24 16 | 38 48 29 27 20 18 47 | 13 45 3  
6 9 3 44 22 | 41 46 39 23 21 26 19 | 33 8 37 28 34 1 10 | 7 25 35 43 6  
2 14  
25 19 43 20 42 10 28 | 47 22 35 5 18 2 4 | 34 40 32 15 9 36 14 | 23 49 3  
8 1 26 24 21 | 29 7 48 33 44 17 30 | 16 3 39 41 12 13 6 | 45 27 11 31 37

46 8  
 13 47 6 36 46 8 2 | 29 27 26 25 21 28 43 | 3 23 49 37 10 42 1 | 15 34  
 7 33 30 14 41 | 4 18 11 35 12 38 40 | 45 22 5 31 32 19 48 | 9 39 24 16 20  
 17 44  
 18 14 35 38 1 49 39 | 23 6 7 15 36 48 44 | 4 13 31 41 22 8 43 | 11 20 1  
 9 16 27 29 46 | 32 24 10 45 5 37 34 | 2 9 25 30 21 17 47 | 26 33 12 3 40  
 28 42  
 48 34 32 37 17 29 16 | 19 30 12 10 1 45 3 | 25 33 44 26 28 5 11 | 43 35 4  
 0 42 39 31 6 | 27 2 22 9 8 13 20 | 14 24 7 38 49 4 46 | 36 15 23 21 18  
 47 41  
 3 9 23 44 22 7 24 | 33 20 11 38 41 8 46 | 39 2 21 12 16 45 35 | 47 17 3  
 2 10 4 18 37 | 14 31 6 25 1 28 43 | 27 29 36 40 15 42 26 | 34 19 13 48 30  
 49 5

---



---

39 17 24 27 8 44 43 | 25 29 28 26 23 41 40 | 31 6 42 36 35 3 2 | 32 37 2  
 1 11 9 16 5 | 18 47 1 13 7 49 33 | 22 45 14 20 30 12 15 | 48 46 34 38 19  
 4 10  
 20 41 11 3 47 28 31 | 17 49 37 19 35 33 7 | 5 32 46 10 45 1 38 | 24 18 2  
 9 26 34 12 13 | 42 15 44 22 30 16 21 | 9 25 4 6 8 48 39 | 14 36 2 40 43  
 23 27  
 1 25 19 46 18 30 7 | 2 4 10 34 5 13 8 | 29 43 11 21 14 9 20 | 44 48 4  
 7 36 28 22 23 | 38 45 12 24 3 6 35 | 17 26 41 27 33 37 40 | 49 32 31 42 16  
 39 15  
 9 32 16 22 45 23 33 | 1 36 18 20 6 11 30 | 8 47 4 7 37 44 49 | 42 15 4  
 6 14 2 41 40 | 48 25 29 39 27 43 28 | 13 19 31 10 38 34 5 | 35 24 17 26 3  
 21 12  
 4 42 10 40 12 34 49 | 22 24 46 16 44 43 45 | 18 15 25 48 27 19 39 | 8 6 3  
 0 31 38 17 7 | 23 20 26 32 36 14 9 | 21 1 11 35 2 28 3 | 29 37 47 5 33  
 41 13  
 38 5 2 26 6 48 15 | 39 42 14 21 12 31 32 | 30 41 16 28 17 33 13 | 27 19 3  
 5 43 20 4 3 | 46 10 34 40 11 8 37 | 47 23 18 49 24 29 36 | 22 9 25 45 44  
 7 1  
 35 29 13 21 37 36 14 | 3 47 38 9 48 15 27 | 12 22 24 23 26 40 34 | 1 25 3  
 9 45 33 49 10 | 17 19 5 4 2 31 41 | 42 16 32 43 44 46 7 | 28 8 20 18 11  
 6 30

---



---

7 46 47 42 48 43 9 | 24 39 44 33 40 12 10 | 13 45 14 38 36 4 41 | 31 32 4  
 9 23 35 6 30 | 34 16 19 21 18 1 26 | 3 37 15 17 5 8 28 | 20 22 29 27 2  
 25 11  
 11 40 15 10 38 45 37 | 46 32 17 48 27 3 41 | 35 49 8 19 30 28 9 | 33 39 1  
 6 13 44 1 29 | 12 36 20 5 22 7 4 | 43 31 2 24 26 6 25 | 21 34 18 14 47  
 42 23  
 26 28 30 34 24 33 35 | 37 9 45 6 29 25 38 | 40 3 48 32 39 23 46 | 19 4 4  
 3 21 11 36 2 | 49 14 17 31 10 47 8 | 18 27 42 44 41 20 22 | 1 12 7 15 5  
 13 16  
 31 36 27 23 2 25 32 | 43 21 13 1 28 5 34 | 7 11 26 33 18 6 17 | 20 8 4  
 1 47 22 45 12 | 24 44 37 46 15 40 42 | 38 49 48 29 14 35 16 | 39 10 19 30 4  
 3 9  
 21 16 20 13 41 18 22 | 15 19 2 14 31 49 36 | 44 27 43 24 1 34 5 | 46 40 1  
 0 38 42 7 9 | 30 23 33 3 25 29 48 | 39 11 47 45 4 32 12 | 6 26 37 17 28  
 8 35  
 17 6 44 14 49 12 29 | 4 7 22 8 26 23 42 | 2 16 15 20 47 25 21 | 5 3 2

8 48 18 37 34 | 43 9 27 11 35 39 45 | 1 30 46 19 13 10 33 | 31 40 36 41 24  
32 38  
5 39 4 8 3 19 1 | 35 11 16 47 30 18 20 | 37 42 12 29 31 10 22 | 17 14 2  
4 25 15 27 26 | 28 6 13 2 41 32 38 | 40 34 21 7 23 36 9 | 33 43 46 49 45  
44 48

---

---

36 1 22 41 21 46 48 | 30 10 39 13 7 42 37 | 43 38 20 40 12 26 33 | 9 44 3  
4 4 17 32 47 | 6 27 2 8 45 24 15 | 11 18 35 3 31 5 19 | 25 29 14 28 23  
16 49  
44 30 7 28 43 26 23 | 16 8 29 18 25 40 21 | 32 5 3 39 41 48 42 | 2 36 3  
1 27 37 20 38 | 22 34 35 47 9 10 1 | 4 12 45 13 46 14 49 | 15 11 33 19 17  
24 6  
40 45 9 49 4 2 5 | 20 43 32 41 15 17 6 | 19 10 13 1 11 21 27 | 16 30 1  
8 22 8 39 14 | 25 12 38 36 33 48 3 | 26 28 24 34 7 23 29 | 42 47 44 35 46  
37 31  
6 12 29 15 10 3 13 | 44 28 24 49 19 47 1 | 45 18 34 16 8 14 4 | 26 23 3  
3 46 5 42 35 | 7 37 31 41 43 11 32 | 25 21 9 48 17 27 20 | 2 38 39 22 36  
30 40  
37 18 42 39 14 17 20 | 27 5 34 46 3 22 33 | 47 44 35 31 25 15 28 | 45 24 4  
8 40 12 11 19 | 21 26 49 16 29 30 23 | 6 32 10 2 36 38 41 | 13 4 1 9 8  
43 7  
32 8 34 33 19 31 27 | 38 14 36 11 4 35 48 | 9 17 6 22 2 24 23 | 10 29 1  
3 7 21 25 49 | 39 28 46 44 40 42 5 | 30 15 16 1 43 47 37 | 41 3 26 20 12  
18 45  
24 35 25 16 11 47 38 | 31 45 23 12 9 26 2 | 49 36 30 46 29 7 37 | 41 43  
3 6 1 28 15 | 19 13 4 17 20 18 14 | 8 40 33 42 22 39 44 | 27 5 10 34 32  
48 21

---

---

47 22 39 1 25 13 34 | 18 2 4 17 20 7 26 | 10 37 36 45 46 41 12 | 49 38 1  
4 15 24 48 43 | 5 8 9 28 32 19 29 | 35 42 3 23 11 21 31 | 16 6 30 44 27  
40 33  
43 38 21 18 20 37 26 | 41 25 33 3 16 6 28 | 42 8 2 9 48 32 40 | 35 22  
5 44 23 47 27 | 36 11 14 1 24 46 31 | 34 13 19 15 45 7 30 | 4 17 49 10 39  
12 29  
15 44 28 4 31 6 42 | 11 35 5 27 43 38 24 | 22 29 23 18 21 49 7 | 12 9 2  
0 17 19 8 45 | 10 30 25 37 26 34 16 | 36 33 40 14 39 41 32 | 47 13 3 46 48  
1 2  
12 23 46 11 35 16 17 | 32 37 42 31 34 30 22 | 28 20 33 14 43 13 15 | 36 1  
6 18 40 21 39 | 47 3 45 27 4 44 7 | 49 48 29 9 10 2 8 | 38 41 5 24 25  
26 19  
41 33 14 29 30 24 45 | 36 15 21 23 13 1 12 | 16 19 27 34 44 38 26 | 3 7 3  
7 2 46 10 11 | 35 48 40 43 39 22 49 | 5 4 28 47 6 25 17 | 32 20 9 8 42  
31 18  
10 7 48 32 40 27 19 | 8 44 49 45 39 9 29 | 11 31 1 5 6 47 3 | 30 42 2  
5 28 13 34 4 | 15 33 41 12 17 21 2 | 46 20 38 18 16 26 24 | 37 14 22 23 35  
36 43  
2 49 3 5 36 9 8 | 10 48 47 40 46 19 14 | 24 35 17 25 4 39 30 | 29 31 2  
6 32 41 33 16 | 20 38 42 18 13 23 6 | 37 44 12 22 27 43 1 | 11 45 21 7 34  
15 28

Sudoku sem valores repetidos

Tempo de execução: 27.6841s

## Adicionar valores inválidos

```
In [ ]: def _():  
        b = box(2)  
        b.add(0,0,10)  
        return  
        _()
```

```
Traceback (most recent call last):  
  File "", line 6, in <module>  
    _()  
  File "", line 3, in _cell_rqHn_  
    b.add(0,0,10)  
  File "", line 15, in add  
    raise ValueError(f"Valor {val} fora de [1, {self.size}]")  
ValueError: Valor 10 fora de [1, 4]
```

## Adicionar valores fora do sudoku

```
In [ ]: def _():  
        b = box(2)  
        b.add(-1,-1,1)  
        return  
        _()
```

```
Traceback (most recent call last):  
  File "", line 5, in <module>  
    _()  
  File "", line 3, in _cell_Gtn0_  
    b.add(-1,-1,1)  
  File "", line 13, in add  
    raise ValueError(f"Célula ({i}, {j}) fora da grelha {self.size}x{self.size}")  
ValueError: Célula (-1, -1) fora da grelha 4x4
```

## Conclusão

Modelamos o Sudoku  $n^2 \times n^2$  como um CSP, com uma abstração única ( `box` ) a partir da qual linhas, colunas, blocos e pistas aleatórias são tratados da mesma forma pelo modelo. Como só a classe do modelo depende do solver, foi possível resolver o problema com o **Z3** e com o **CP-SAT**, alterando apenas essa classe. O fluxo completo (pistas aleatórias, montagem dos grupos, resolução e validação) funcionou para  $n = 1$  até  $n = 7$ . Quando as pistas aleatórias não têm solução, simplesmente dizemos que não tem solução. Quanto ao desempenho, CP-SAT foi consideravelmente muito mais rápido que o Z3.